Zcim Project

CP/M® Dynamic Debugging Tool (DDT)

User's Guide

version 2025-08-02

Printing

Zcim Project edition: version 2025-08-02

Contents

List of Tables

1. Introduction

The DDT program allows dynamic interactive testing and debugging of programs generated in the CP/M environment. Invoke the debugger with a command of one of the following forms:

```
DDT
DDT filename.HEX
DDT filename.COM
```

where *filename* is the name of the program to be loaded and tested. In both cases, the DDT program is brought into main memory in place of the Console Command Processor (CCP) and resides directly below the Basic Disk Operating System (BDOS) portion of CP/M. Refer to Section 5 for standard memory organization. The BDOS starting address, located in the address field of the JMP instruction at location 0005H, is altered to reflect the reduced Transient Program Area (TPA) size.

The second and third forms of the DDT command perform the same actions as the first, except there is a subsequent automatic load of the specified HEX or COM file. The action is identical to the following sequence of commands:

```
DDT
Ifilename.HEX or Ifilename.COM
R
```

where the I and R commands set up and read the specified program to test. See the explanation of the I and R commands below for exact details.

Upon initiation, DDT prints a sign-on message in the form:

```
nnK DDT-S VER m.m
```

where *nn* is the memory size (which much match the CP/M system being used), *S* is the hardware system which is assumed, corresponding to the codes

D Digital Research standard version
M MDS version
I IMSAI standard version
O Omron systems
S Digital Systems standard version

and *m.m* is the revision number.

Following the sign-on message, DDT prompts the operator with the hyphen character, -, and waits for input commands from the console. The operator can type any of several single character commands, terminated by a carriage return to execute the command. Each line of input can be line-edited using the following standard CP/M controls:

Character	Meaning
rubout/DEL	remove the last character typed
CTRL-U	remove the entire line, ready for re-typing
CTRL-C	system reboot

Table 1-1: DDT Line-editing Controls

Any command can be up to 32 characters in length (an automatic carriage return is inserted as character 33), where the first character determines the command type. Table 1-2 describes DDT commands.

Character	Result
A	enters assembly-language mnemonics with operands.
D	displays memory in hexadecimal and ASCII.
F	fills memory with constant data.
G	begins execution with optional breakpoints.
I	sets up a standard input File Control Block.
L	lists memory using assembler mnemonics.
M	moves a memory segment from source to destination.
R	reads a program for subsequent testing.
S	substitutes memory values.
T	traces program execution.
U	untraced program monitoring.
X	examines and optionally alters the CPU state.

Table 1-2: DDT Commands

The command character, in some cases, is followed by zero, one, two, or three hexadecimal values, which are separated by commas or single blank characters. All DDT numeric output is in hexadecimal form. The commands are not executed until the carriage return is typed at the end of the command.

At any point in the debug run, you can stop execution of DDT by using either a CTRL-C or G0 (jump to location 0000H) and save the current memory image by using a SAVE command of the form:

SAVE *n filename*.COM

where n is the number of pages (256 byte blocks) to be saved on disk. The number of blocks is determined by taking the high-order byte of the address in the TPA and converting this number to decimal. For example, if the highest address in the TPA is 134H, the number of pages is 12H or 18 in decimal. You could type a CTRL-C during the debug run, returning to the CCP level, followed by

```
SAVE 18 X.COM
```

The memory image is saved as X.COM on the disk and can be directly executed by typing the name X. If further testing is required, the memory image can be recalled by typing

```
DDT X.COM
```

which reloads the previously saved program from location 100H through page 18, 23FFH. The CPU state is not a part of the COM file; thus, the program must be restarted from the beginning to test it properly.

2. DDT Commands

The individual commands are detailed below. In each case, the operator must wait for the hyphen prompt character before entering the command. If control is passed to a program under test, and the program has not reached a breakpoint, control can be returned to DDT by executing a RST 7 from the front panel. In the explanation of each command, the command letter is shown in some cases with numbers separated by commas, the numbers are represented by lower-case letters. These numbers are always assumed to be in a hexadecimal radix and from one to four digits in length. Longer numbers are automatically truncated on the right.

Many of the commands operate upon a “CPU state” that corresponds to the program under test. The CPU state holds the registers of the program being debugged and initially contains zeros for all registers and flags except for the program counter, P, and stack pointer, S, which default to 100H. The program counter is subsequently set to the starting address given in the last record of a HEX file if a file of this form is loaded, see the I and R commands.

2.1. The A (Assembly) Command

DDT allows in-line assembly language to be inserted into the current memory image using the A command, which takes the form:

A*s*

where *s* is the hexadecimal starting address for the in-line assembly. DDT prompts the console with the address of the next instruction to fill and reads the console, looking for assembly-language mnemonics followed by register references and operands in absolute hexadecimal form. See the Intel 8080 Assembly Language Reference Card for a list of mnemonics. Each successive load address is printed before reading the console. The A command terminates when the first empty line is input from the console.

Upon completion of assembly language input, you can review the memory segment using the DDT disassembler (see the L command).

Note that the assembler/disassembler portion of DDT can be overlaid by the transient program being tested, in which case the DDT program responds with an error condition when the A and L commands are used.

2.2. The D (Display) Command

The D command allows you to view the contents of memory in hexadecimal and ASCII formats. The D command takes the forms:

```
D
Ds
Ds,f
```

In the first form, memory is displayed from the current display address, initially 100H, and continues for 16 display lines. Each display line takes the following form:

```
aaaa bb bb bb bb bb bb bb bb bb bb bb bb bb bb bb bb ccccccccccccccc
```

where *aaaa* is the display address in hexadecimal and *bb* represents data present in memory starting at *aaaa*. The ASCII characters starting at *aaaa* are to the right (represented by the sequence of character *c*) where non-graphic characters are printed as a period. You should note that both upper- and lower-case alphabetic characters are displayed, and will appear as upper-case symbols on a console device that supports only upper-case. Each display line gives the values of 16 bytes of data, with the first line truncated so that the next line begins at an address that is a multiple of 16.

The second form of the D command is similar to the first, except that the display address is first set to address *s*.

The third form causes the display to continue from address *s* through address *f*. In all cases, the display address is set to the first address not displayed in this command, so that a continuing display can be accomplished by issuing successive D commands with no explicit addresses.

Excessively long displays can be aborted by pressing the return key.

2.3. The F (Fill) Command

The F command takes the form:

F*s*,*f*,*c*

where *s* is the starting address, *f* is the final address, and *c* is a hexadecimal byte constant. DDT stores the constant *c* at address *s*, increments the value of *s* and test against *f*. If *s* exceeds *f*, the operation terminates, otherwise the operation is repeated. Thus, the fill command can be used to set a memory block to a specific constant value.

2.4. The G (Go) Command

A program is executed using the G command, with up to two optional breakpoint addresses. The G command takes the forms:

G

G*s*

G*s*,*b*

G*s*,*b*,*c*

G,*b*

G,*b*,*c*

The first form executes the program at the current value of the program counter in the current machine state, with no breakpoints set. The only way to regain control in DDT is through a RST 7 execution. The current program counter can be viewed by typing an X or XP command.

The second form is similar to the first, except that the program counter in the current machine state is set to address *s* before execution begins.

The third form is the same as the second, except that program execution stops when address *b* is encountered (*b* must be in the area of the program under test). The instruction at location *b* is not executed when the breakpoint is encountered.

The fourth form is identical to the third, except that two breakpoints are specified, one at *b* and the other at *c*. Encountering either breakpoint causes execution to stop and both breakpoints are cleared. The last two forms take the program counter from the current machine state and set one and two breakpoints, respectively.

Execution continues from the starting address in real-time to the next breakpoint. There is no intervention between the starting address and the break address by DDT. If the program under test does not reach a breakpoint, control cannot return to DDT without executing a RST 7 instruction. Upon encountering a breakpoint, DDT stops execution and types

**d*

where *d* is the stop address. The machine state can be examined at this point using the X (Examine) command. You must specify breakpoints that differ from the program counter address at the beginning of the G command. Thus, if the current program counter is 1234H, then the following commands:

```
G,1234
```

and

```
G400,400
```

both produce an immediate breakpoint without executing any instructions.

2.5. The I (Input) Command

The I command allows you to insert a filename into the default File Control Block (FCB) at 5CH. The FCB created by CP/M for transient programs is placed at this location (see the CP/M Interface Guide). The default FCB can be used by the program under test as if it had been passed by the CP/M Console Processor. Note that this filename is also used by DDT for reading additional HEX and COM files. The I command takes the forms:

I*filename*

or

I*filename*.*filetype*

If the second form is used and the filetype is either HEX or COM, subsequent R commands can be used to read the pure binary or hex format machine code. Section 2.8 gives further details.

2.6. The L (List) Command

The L command is used to list assembly-language mnemonics in a particular program region. The L command takes the forms:

L

L*s*

L*s*,*f*

The first form lists twelve lines of disassembled machine code from the current list address. The second form sets the list address to *s* and then lists twelve lines of code. The last form lists disassembled code from *s* through address *f*. In all three cases, the list address is set to the next unlisted location in preparation for a subsequent L command. Upon encountering an execution breakpoint, the list address is set to the current value of the program counter (G and T commands). Again, long type-outs can be aborted by pressing RETURN during the list process.

2.7. The M (Move) Command

The M command allows block movement of program or data areas from one location to another in memory. The M command takes the form:

```
MS,f,d
```

where *s* is the start address of the move, *f* is the final address, and *d* is the destination address. Data is first removed from *s* to *d*, and both addresses are incremented. If *s* exceeds *f*, the move operation stops; otherwise, the move operation is repeated.

2.8. The R (Read) Command

The R command is used in conjunction with the I command to read COM and HEX files from the disk into the transient program area in preparation for the debug run. The R command takes the forms:

```
R
```

```
Rb
```

where *b* is an optional bias address that is added to each program or data address as it is loaded. The load operation must not overwrite any of the system parameters from 000H through 0FFH (that is, the first page of memory). If *b* is omitted, then $b = 0000$ is assumed. The R command requires a previous I command, specifying the name of a HEX or COM file. The load address for each record is obtained from each individual HEX record, while an assumed load address of 100H is used for COM files. Note that any number of R commands can be issued following the I command to reread the program under test, assuming the tested program does not destroy the default area at 5CH. Any file specified with the filetype COM is assumed to contain machine code in pure binary form (created with the LOAD or SAVE command), and all others are assumed to contain machine code in Intel hex format (produced, for example, with the ASM command).

Recall that the command,

```
DDT  filename.typ
```

which initiates the DDT program, equals to the following commands:

```
DDT
-filename.typ
-R
```

Whenever the R command is issued, DDT responds with either the error indicator ? (file cannot be opened, or a checksum error occurred in a HEX file) or with a load message. The load message takes the form:

```
NEXT PC
nnnn pppp
```

where *nnnn* is the next address following the loaded program and *pppp* is the assumed program counter (100H for COM files, or taken from the last record if a HEX file is specified).

2.9. The S (Set) Command

The S command allows memory locations to be examined and optionally altered. The S command takes the form:

Ss

where s is the hexadecimal starting address for examination and alteration of memory. DDT responds with a numeric prompt, giving the memory location, along with the data currently held in memory. If you type a carriage return, the data is not altered. If a byte value is typed, the value is stored at the prompted address. In either case, DDT continues to prompt with successive addresses and values until you type either a period or an invalid input value is detected.

2.10. The T (Trace) Command

The T command allows selective tracing of program execution for 1 to 65535 program steps. The T command takes the forms:

T

T*n*

In the first form, the CPU state is displayed and the next program step is executed. The program terminates immediately, with the termination address displayed as

**hhhh*

where *hhhh* is the next address to execute. The display address (used in the D command) is set to the value of H and L, and the list address (used in the L command) is set to *hhhh*. The CPU state at program termination can then be examined using the X command.

The second form of the T command is similar to the first, except that execution is traced for *n* steps (*n* is a hexadecimal value) before a program breakpoint occurs. A breakpoint can be forced in the trace mode by typing a rubout character. The CPU state is displayed before each program step is taken in trace mode. The format of the display is the same as described in the X command.

You should note that program tracing is discontinued at the CP/M interface and resumes after return from CP/M to the program under test. Thus, CP/M functions that access I/O devices, such as the disk drive, run in real-time, avoiding I/O timing problems. Programs running in trace mode execute approximately 500 times slower than real-time because DDT gets control after each user instruction is executed. Interrupt processing routines can be traced, but commands that use the breakpoint facility (G, T, and U) accomplish the break using an RST 7 instruction, which means that the tested program cannot use this interrupt location. Further, the trace mode always runs the tested program with interrupts enabled, which may cause problems if asynchronous interrupts are received during tracing.

To get control back to DDT during trace, press RETURN rather than executing an RST 7. This ensures that the trace for current instruction is completed before interruption.

2.11. The U (Untrace) Command

The U command is identical to the T command, except that intermediate program steps are not displayed. The untrace mode allows from 1 to 65535 (0FFFFH) steps to be executed in monitored mode and is used principally to retain control of an executing program while it reaches steady state conditions. All conditions of the T command apply to the U command.

2.12. The X (Examine) Command

The X command allows selective display and alteration of the current CPU state for the program under test. The X command takes the forms:

X

X*r*

where *r* is one of the 8080 CPU registers listed in Table 2-3.

Register	Meaning	Value
C	Carry flag	0 or 1
Z	Zero flag	0 or 1
M	Minus flag	0 or 1
E	Even parity flag	0 or 1
I	Half-carry	0 or 1
A	Accumulator	0 ... FF
B	BC register pair	0 ... FFFF
D	DE register pair	0 ... FFFF
H	HL register pair	0 ... FFFF
S	Stack pointer	0 ... FFFF
P	Program counter	0 ... FFFF

Table 2-3: CPU Registers

In the first case, the CPU register state is displayed in the format:

C*f* **Z***f* **M***f* **E***f* **I***f* **A=***bb* **B=***dddd* **D=***dddd* **H=***dddd* **S=***dddd* **P=***dddd* *inst*

where *f* is a 0 or 1 flag value, *bb* is a byte value, and *dddd* is a double-byte quantity corresponding to the register pair. The *inst* field contains the disassembled instruction, that occurs at the location addressed by the CPU state's program counter.

The second form allows display and optional alteration of register values, where r is one of the registers given above (C, Z, M, E, I, A, B, D, H, S, or P). In each case, the flag or register value is first displayed at the console. The DDT program then accepts input from the console. If a carriage return is typed, the flag or register value is not altered. If a value in the proper range is typed, the flag or register value is altered. You should note that BC, DE, and HL are displayed as register pairs. Thus, you must type the entire register pair when B, C, or the BC pair is altered.

3. Implementation Notes

The organization of DDT allows certain nonessential portions to be overlaid to gain a larger transient program area for debugging large programs. The DDT program consists of two parts: the DDT nucleus and the assembler/disassembler module. The DDT nucleus is loaded over the CCP and, although loaded with the DDT nucleus, the assembler/disassembler can be overlaid unless used to assemble or disassemble.

In particular, the BDOS address at location 0006H (address field of the JMP instruction at location 0005H) is modified by DDT to address the base location of the DDT nucleus, which, in turn, contains a JMP instruction to the BDOS. Thus, programs that use this address field to size memory see the logical end of memory at the base of the DDT nucleus rather than the base of the BDOS.

The assembler/disassembler module resides directly below the DDT nucleus in the transient program area. If the A, L, T, or X commands are used during the debugging process, the DDT program again alters the address field at 0006H to include this module, further reducing the logical end of memory. If a program loads beyond the beginning of the assembler/disassembler module, the A and L commands are lost (their use produces a ? in response) and the trace and display (T and X) commands list the inst field of the display in hexadecimal, rather than as a decoded instruction.

4. An Example

The following example shows an edit, assemble, and debug for a simple program that reads a set of data values and determines the largest value in the set. The largest value is taken from the vector and stored into LARGE at the termination of the program.

In the example:

output text `is in a monospace font`

input text `is in purple`

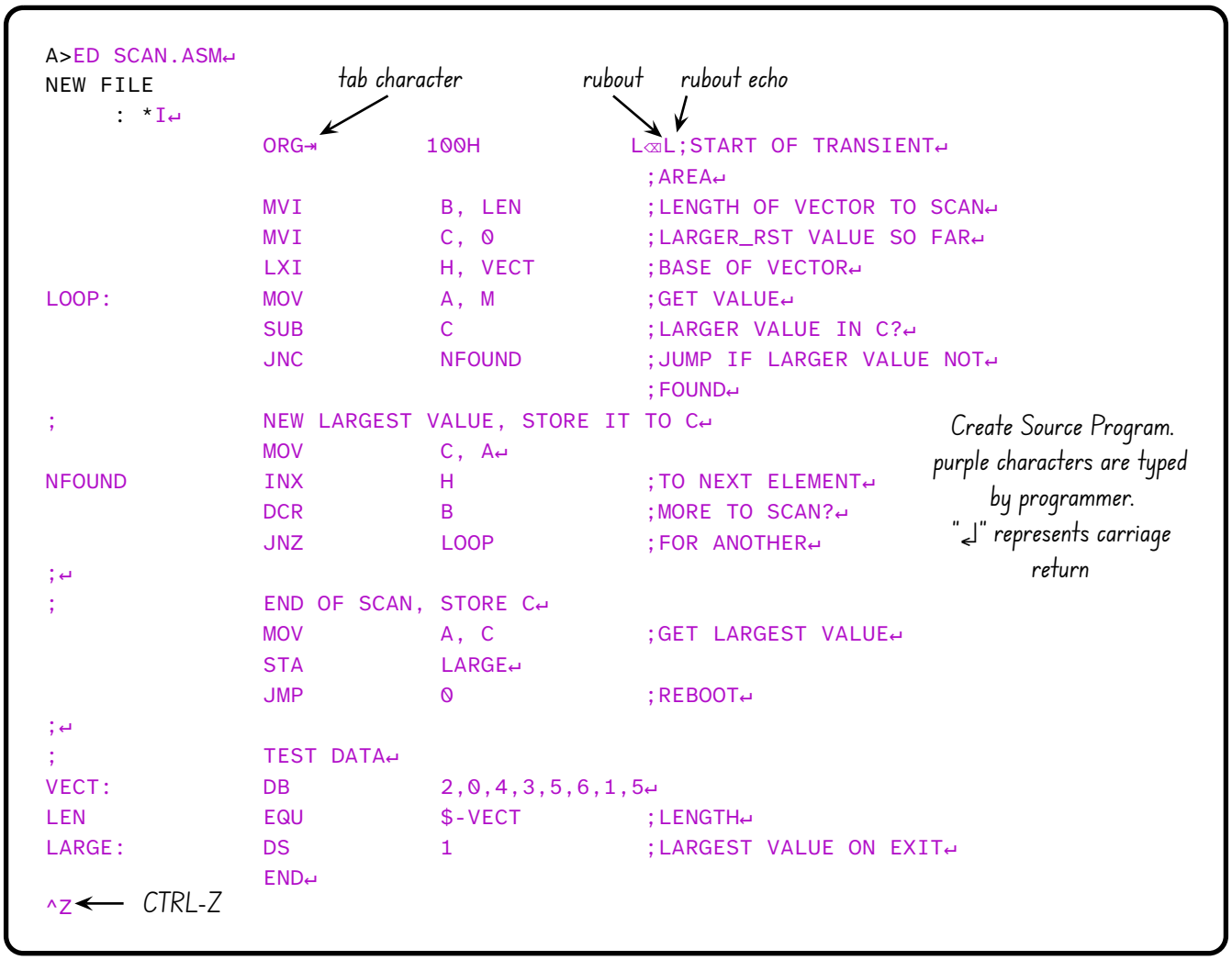
carriage-return is displayed as ↲

rubout/DEL key is displayed as ⌫

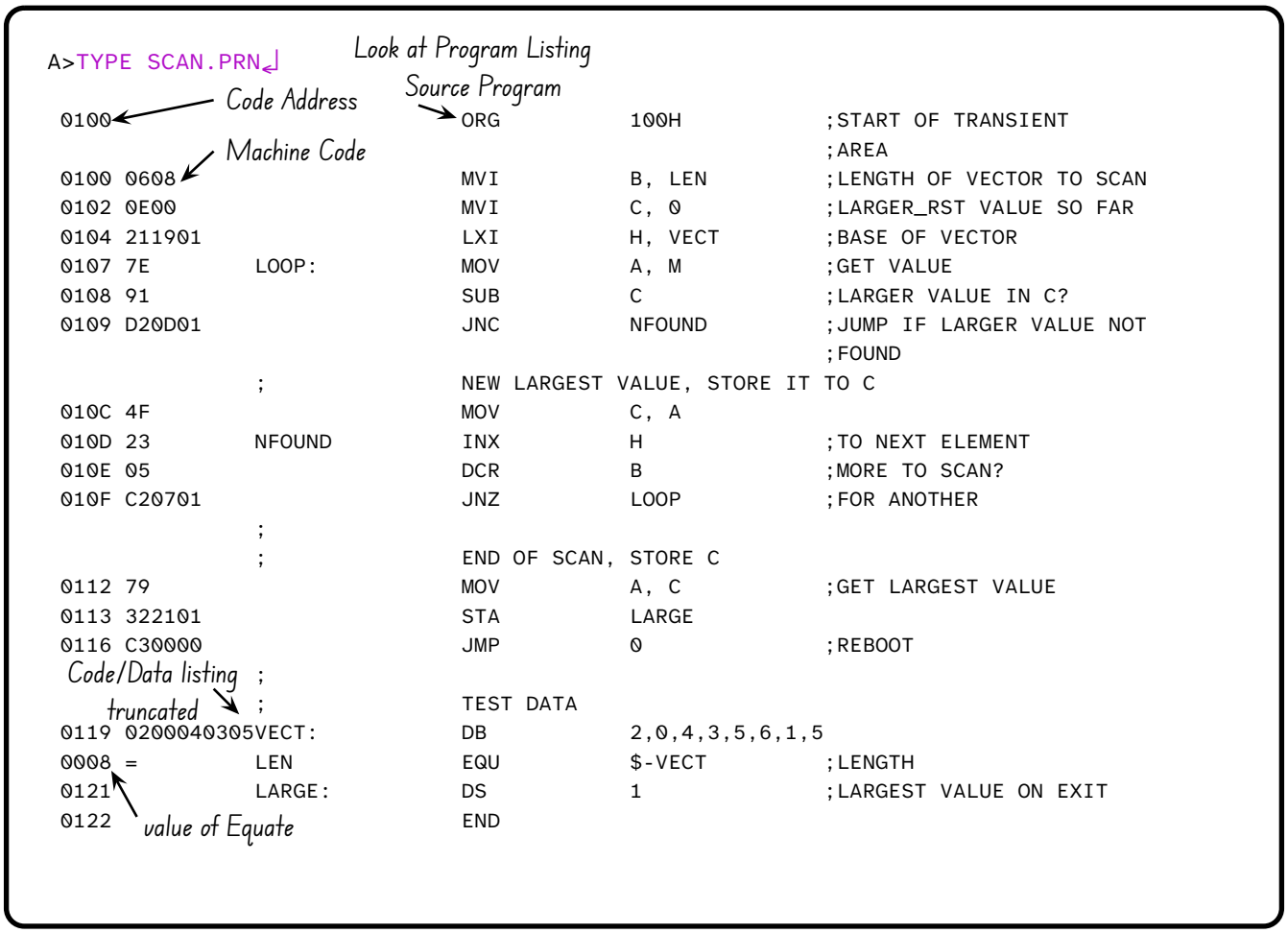
tab key is displayed as →|

CTRL-Z ^Z

comments *are displayed in 'hand-written' text.*



```
    : *B0P↵
   1:                   ORG          100H           ;START OF TRANSIENT
   2:                                               ;AREA
   3:                   MVI          B, LEN         ;LENGTH OF VECTOR TO SCAN
   4:                   MVI          C, 0           ;LARGER_RST VALUE SO FAR
   5:                   LXI          H, VECT        ;BASE OF VECTOR
   6:   LOOP:           MOV          A, M           ;GET VALUE
   7:                   SUB          C              ;LARGER VALUE IN C?
   8:                   JNC          NFOUND         ;JUMP IF LARGER VALUE NOT
   9:                                               ;FOUND
  10:   ;               NEW LARGEST VALUE, STORE IT TO C
  11:                   MOV          C, A
  12:   NFOUND          INX          H              ;TO NEXT ELEMENT
  13:                   DCR          B              ;MORE TO SCAN?
  14:                   JNZ          LOOP           ;FOR ANOTHER
  15:   ;
  16:   ;               END OF SCAN, STORE C
  17:                   MOV          A, C           ;GET LARGEST VALUE
  18:                   STA          LARGE
  19:                   JMP          0              ;REBOOT
  20:   ;
  21:   ;               TEST DATA
  22:   VECT:           DB           2,0,4,3,5,6,1,5
  23:   LEN             EQU          $-VECT         ;LENGTH
   1: *E↵  ← End of Edit

A>ASM SCAN↵        Start Assembler
CP/M ASSEMBLER - VER 1.0

0122
002H USE FACTOR
END OF ASSEMBLY       Assembly Complete - Look at Program Listing
```


A>TYPE SCAN.PRN↵
Look at Program Listing
Source Program
Code Address
Machine Code
0100
0100 0608
0102 0E00
0104 211901
0107 7E
0108 91
0109 D20D01
010C 4F
010D 23
010E 05
010F C20701
0112 79
0113 322101
0116 C30000
0119 0200040305
0008 =
0121
0122
LOOP:
NFOUND
VECT:
LEN
LARGE:
ORG 100H ;START OF TRANSIENT
;AREA
MVI B, LEN ;LENGTH OF VECTOR TO SCAN
MVI C, 0 ;LARGER_RST VALUE SO FAR
LXI H, VECT ;BASE OF VECTOR
MOV A, M ;GET VALUE
SUB C ;LARGER VALUE IN C?
JNC NFOUND ;JUMP IF LARGER VALUE NOT
;FOUND
; NEW LARGEST VALUE, STORE IT TO C
MOV C, A
INX H ;TO NEXT ELEMENT
DCR B ;MORE TO SCAN?
JNZ LOOP ;FOR ANOTHER
;
; END OF SCAN, STORE C
MOV A, C ;GET LARGEST VALUE
STA LARGE
JMP 0 ;REBOOT
Code/Data listing truncated
; TEST DATA
DB 2,0,4,3,5,6,1,5
EQU $-VECT ;LENGTH
DS 1 ;LARGEST VALUE ON EXIT
END
value of Equate

```
A>DDT SCAN.HEX↵          Start Debugger with hex format machine code

16K DDT VERS 1.0
NEXT PC
0121 0000
-X↵          last load address + 1                    next instruction to execute
                                                          at PC = 0
C0Z0M0E0I0 A=00 B=0000 D=0000 H=0000 S=0100 P=0000 JMP   FA03
        examine registers before debug run
-XP↵
P=0000 100↵      change PC to 100
-X↵          look at registers again

                                          PC changed
C0Z0M0E0I0 A=00 B=0000 D=0000 H=0000 S=0100 P=0100 MVI B,08
                                                   next instruction
-L100↵                                             to execute at PC=100

  0100  MVI  B,08
  0102  MVI  C,00
  0104  LXI  H,0119
  0107  MOV  A,M
  0108  SUB  C
  0109  JNC  010D
  010C  MOV  C,A
  010D  INX  H
  010E  DCR  B
  010F  JNZ  0107
  0112  MOV  A,C
-L↵

  0113  STA  0121
  0116  JMP  0000
  0119  STAX B
  011A  NOP
  011B  INR  B
  011C  INX  B
  011D  DCR  B
  011E  MVI  B,01
  0120  DCR  B
  0121  ??=  20
  0122  MOV  D,D
```

Diassembled machine code at 100H (See source listing for comparison)

A little more machine code (note that program ends at location 116 with a JMP to 0000)

```
-A116↵                enter inline assembly mode to change the JMP to 0000 into a RST 7, which will
0116 RST 7↵           cause the program under test to return to DDT if 116H is ever executed
0117 ↵                (single carriage return stops assemble mode)
-L113↵
```

list code at 113H to check that RST 7 was properly inserted in place of JMP

```
  0113  STA  0121
  0116  RST  07
  0117  NOP
  0118  NOP
  0119  STAX B
  011A  NOP
  011B  INR  B
  011C  INX  B
  011D  DCR  B
  011E  MVI  B,01
  0120  DCR  B
-X↵           look at registers

C0Z0M0E0I0 A=00 B=0000 D=0000 H=0000 S=0100 P=0100 MVI B,08

-T↵           execute program for one step
```

initial CPU state, before instruction is executed

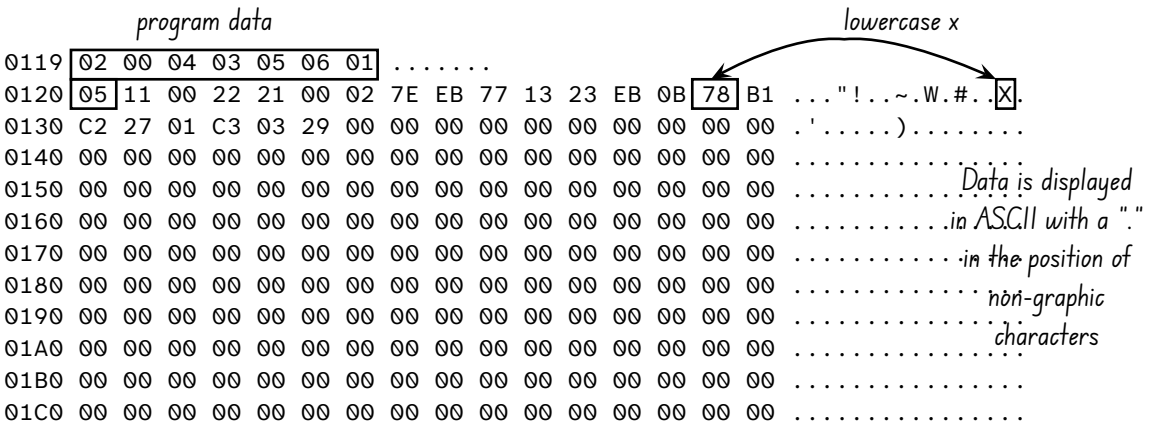
```
C0Z0M0E0I0 A=00 B=0000 D=0000 H=0000 S=0100 P=0100 MVI B,08*0102

-T↵           trace one step again (note 08H in B)
```

automatic breakpoint

```
C0Z0M0E0I0 A=00 B=0800 D=0000 H=0000 S=0100 P=0102 MVI C,00*0104

-T↵           trace again (register C is cleared)

C0Z0M0E0I0 A=00 B=0800 D=0000 H=0000 S=0100 P=0104 LXI H,0119*0107

-T3↵          trace three steps

C0Z0M0E0I0 A=00 B=0800 D=0000 H=0119 S=0100 P=0107 MOV A,M
C0Z0M0E0I0 A=02 B=0800 D=0000 H=0119 S=0100 P=0108 SUB C
C0Z0M0E0I1 A=02 B=0800 D=0000 H=0119 S=0100 P=0109 JNC 010D*010D
```

automatic breakpoint at 10DH

```
-D119↵      display memory starting at 119H
```




```
-X↵         current CPU state

C0Z0M0E0I0 A=02 B=0800 D=0000 H=0119 S=0100 P=010D INX  H

-T5↵        trace 5 steps from current CPU state

C0Z0M0E0I0 A=02 B=0800 D=0000 H=0119 S=0100 P=010D INX  H
C0Z0M0E0I0 A=02 B=0800 D=0000 H=011A S=0100 P=010E DCR  B
C0Z0M0E0I0 A=02 B=0700 D=0000 H=011A S=0100 P=010F JNZ  0107
C0Z0M0E0I0 A=02 B=0700 D=0000 H=011A S=0100 P=0107 MOV  A,M
C0Z0M0E0I0 A=00 B=0700 D=0000 H=011A S=0100 P=0108 SUB  C*0109
-U5↵        trace without listing intermediate states
```

automatic breakpoint

```
C0Z1M0E0I0 A=00 B=0700 D=0000 H=011A S=0100 P=0109 JNC  010D*0108
-X↵         CPU state at end of U5

C0Z0M0E0I0 A=04 B=0600 D=0000 H=011B S=0100 P=0108 SUB  C

-G↵         run program from current PC until completion (in real-time)
*0116       breakpoint at 116H, caused by executing RST 7 in machine code
-X↵         CPU state at end of program

C0Z1M0E0I0 A=00 B=0000 D=0000 H=0121 S=0100 P=0116 RST  07

-XP↵        examine and change program counter
P=0116 100↵
-X↵

C0Z1M0E0I0 A=00 B=0000 D=0000 H=0121 S=0100 P=0100 MVI  B,08
```

```
-T10↵
C0Z1M0E0I0 A=00 B=0000 D=0000 H=0121 S=0100 P=0100 MVI  B,08
C0Z1M0E0I0 A=00 B=0800 D=0000 H=0121 S=0100 P=0102 MVI  C,00
C0Z1M0E0I0 A=00 B=0800 D=0000 H=0121 S=0100 P=0104 LXI  H,0119
C0Z1M0E0I0 A=00 B=0800 D=0000 H=0119 S=0100 P=0107 MOV  A,M
C0Z1M0E0I0 A=02 B=0800 D=0000 H=0119 S=0100 P=0108 SUB  C
C0Z0M0E0I0 A=02 B=0800 D=0000 H=0119 S=0100 P=0109 JNC  010D
C0Z0M0E0I0 A=02 B=0800 D=0000 H=0119 S=0100 P=010D INX  H
C0Z0M0E0I0 A=02 B=0800 D=0000 H=011A S=0100 P=010E DCR  B
C0Z0M0E0I0 A=02 B=0700 D=0000 H=011A S=0100 P=010F JNZ  0107
C0Z0M0E0I0 A=02 B=0700 D=0000 H=011A S=0100 P=0107 MOV  A,M
C0Z0M0E0I0 A=00 B=0700 D=0000 H=011A S=0100 P=0108 SUB  C
C0Z1M0E0I0 A=00 B=0700 D=0000 H=011A S=0100 P=0109 JNC  010D
C0Z1M0E0I0 A=00 B=0700 D=0000 H=011A S=0100 P=010D INX  H
C0Z1M0E0I0 A=00 B=0700 D=0000 H=011B S=0100 P=010E DCR  B
C0Z0M0E0I0 A=00 B=0600 D=0000 H=011B S=0100 P=010F JNZ  0107
C0Z0M0E0I0 A=00 B=0600 D=0000 H=011B S=0100 P=0107 MOV  A,M*0108
```

trace 10 (hexadecimal) steps

first data element

current largest value

subtract for comparison A < C

Program should have moved the value from A into C since A > C. Since this case was not executed, it appears that the JNC should have been a JC instruction

```
-A109↵
-JC 10D↵
010C↵
-G0↵
A>SAVE 1 SCAN.COM↵
A>DDT SCAN.COM↵

DDT VERS 2.2
NEXT PC
0200 0100

-L100↵

  0100  MVI   B,08
  0102  MVI   C,00
  0104  LXI   H,0119
  0107  MOV   A,M
  0108  SUB   C
  0109  JC    010D
  010C  MOV   C,A
  010D  INX   H
  010E  DCR   B
  010F  JNZ   0107
  0112  MOV   A,C

-XP↵
P=0100 ↵
```

Insert a "hot patch" into the machine code to change the JNC to JC

stop DDT so that a version of the patched program can be saved

program resides on first page, so save 1 page

restart DDT with the saved memory image to continue testing

list some code

previous patch is present in SCAN.COM

```
-T10↲      trace to see how patched version operates
                                        data is moved from A to C
C0Z0M0E0I0 A=00 B=0000 D=0000 H=0000 S=0100 P=0100 MVI B,08
C0Z0M0E0I0 A=00 B=0800 D=0000 H=0000 S=0100 P=0102 MVI C,00
C0Z0M0E0I0 A=00 B=0800 D=0000 H=0000 S=0100 P=0104 LXI H,0119
C0Z0M0E0I0 A=00 B=0800 D=0000 H=0119 S=0100 P=0107 MOV A,M
C0Z0M0E0I0 A=02 B=0800 D=0000 H=0119 S=0100 P=0108 SUB C
C0Z0M0E0I1 A=02 B=0800 D=0000 H=0119 S=0100 P=0109 JC  010D
C0Z0M0E0I1 A=02 B=0800 D=0000 H=0119 S=0100 P=010C MOV C,A
C0Z0M0E0I1 A=02 B=0802 D=0000 H=0119 S=0100 P=010D INX H
C0Z0M0E0I1 A=02 B=0802 D=0000 H=011A S=0100 P=010E DCR B
C0Z0M0E0I1 A=02 B=0702 D=0000 H=011A S=0100 P=010F JNZ 0107
C0Z0M0E0I1 A=02 B=0702 D=0000 H=011A S=0100 P=0107 MOV A,M
C0Z0M0E0I1 A=00 B=0702 D=0000 H=011A S=0100 P=0108 SUB C
C1Z0M1E0I0 A=FE B=0702 D=0000 H=011A S=0100 P=0109 JC  010D
C1Z0M1E0I0 A=FE B=0702 D=0000 H=011A S=0100 P=010D INX H
C1Z0M1E0I0 A=FE B=0702 D=0000 H=011B S=0100 P=010E DCR B
C1Z0M0E1I1 A=FE B=0602 D=0000 H=011B S=0100 P=010F JNZ 0107*0107
                                        breakpoint after 16 steps
-X↲
C1Z0M0E1I1 A=FE B=0602 D=0000 H=011B S=0100 P=0107 MOV A,M

-G,108↲   run from current PC and breakpoint at 108H
*0108
-X↲              next data item
C1Z0M0E1I1 A=04 B=0602 D=0000 H=011B S=0100 P=0108 SUB C

-T↲                      single step for a few cycles
C1Z0M0E1I1 A=04 B=0602 D=0000 H=011B S=0100 P=0108 SUB C*0109

-T↲
C0Z0M0E0I1 A=02 B=0602 D=0000 H=011B S=0100 P=0109 JC 010D*010C

-X↲
C0Z0M0E0I1 A=02 B=0602 D=0000 H=011B S=0100 P=010C MOV C,A

-G↲       run to completion
*0116
-X↲
C0Z1M0E1I1 A=03 B=0003 D=0000 H=0121 S=0100 P=0116 RST 07

-S121↲       look at the value of "LARGE"
 0121   03  ↲        Wrong value!
 0122   52  .↲
```

```
-L100↵

  0100  MVI   B,08
  0102  MVI   C,00
  0104  LXI   H,0119
  0107  MOV   A,M
  0108  SUB   C
  0109  JC    010D
  010C  MOV   C,A
  010D  INX   H
  010E  DCR   B
  010F  JNZ   0107
  0112  MOV   A,C
-L↵

  0113  STA   0121
  0116  RST   07
  0117  NOP
  0118  NOP
  0119  STAX  B
  011A  NOP
  011B  INR   B
  011C  INX   B
  011D  DCR   B
  011E  MVI   B,01
  0120  DCR   B
-XP↵
```

Review the code

reset the PC

```
P=0116 100↵
```

single step, and watch data values

```
-T↵

C0Z1M0E1I1 A=03 B=0003 D=0000 H=0121 S=0100 P=0100 MVI B,08*0102

-T↵
```

count set

```
C0Z1M0E1I1 A=03 B=0803 D=0000 H=0121 S=0100 P=0102 MVI C,00*0104
-T↵
```

"largest" set

```
C0Z1M0E1I1 A=03 B=0800 D=0000 H=0121 S=0100 P=0104 LXI H,0119*0107
-T↵
```

base address of data set

```
C0Z1M0E1I1 A=03 B=0800 D=0000 H=0119 S=0100 P=0107 MOV A,M*0108
-T↵
```

first data item brought to A

```
C0Z1M0E0I0 A=02 B=0800 D=0000 H=0119 S=0100 P=0108 SUB  C*0109
```

```
-T↲
C0Z0M0E0I0 A=02 B=0800 D=0000 H=0119 S=0100 P=0109 JC   010D*010C
-T↲
C0Z0M0E0I0 A=02 B=0800 D=0000 H=0119 S=0100 P=010C MOV  C,A*010D
-T↲                                  first data item moved to C correctly
C0Z0M0E0I1 A=02 B=0802 D=0000 H=0119 S=0100 P=010D INX H*010E
-T↲
C0Z0M0E0I1 A=02 B=0802 D=0000 H=011A S=0100 P=010E DCR B*010F
-T↲
C0Z0M0E0I1 A=02 B=0702 D=0000 H=011A S=0100 P=010F JNZ 0107*0107
-T↲
C0Z0M0E0I1 A=02 B=0702 D=0000 H=011A S=0100 P=0107 MOV A,M*0108
-T↲                                  second data item brought to A
C0Z0M0E0I1 A=00 B=0702 D=0000 H=011A S=0100 P=0108 SUB C*0109
-T↲                                  subtract destroys data value which was loaded!
C1Z0M1E0I0 A=FE B=0702 D=0000 H=011A S=0100 P=0109 JC 010D*010D
-T↲
C1Z0M1E0I0 A=FE B=0702 D=0000 H=011A S=0100 P=010D INX H*010E
-L100↲
 0100   MVI         B,08
 0102   MVI         C,00
 0104   LXI         H,0119
 0107   MOV         A,M
 0108   SUB         C          <-- This should have been a CMP so that register A
 0109   JC          010D           would not be changed
 010C   MOV         C,A
 010D   INX         H
 010E   DCR         B
 010F   JNZ         0107
 0112   MOV         A,C
-A108↲
0108 CMP C↲        hot patch at 108H changes SUB to CMP
0109 ↲
-G0↲               stop DDT for Save
A>SAVE 1 SCAN.COM↲     save memory image
```

```
A>DDT SCAN.COM↲    restart DDT

DDT VERS 2.2
NEXT PC
0200 0100

-XP↲
P=0100 ↲
-L116↲
 0116   RST         07
 0117   NOP
 0118   NOP
 0119   STAX        B
 011A   NOP
```

look at code to see if it was properly loaded (long typeout aborted with rubout)

```
⌫
-G,116↲          run from 100H to completion


*0116

-XC↲             look at Carry (accidental typo)
C1 ↲
-X↲              look at CPU state
C1Z1M0E1I1 A=06 B=0006 D=0000 H=0121 S=0100 P=0116 RST 07

-S121↲             look at "LARGE" - it appears to be correct
 0121   06  ↲
 0122   00  ↲
 0123   22  .↲
-G0↲               stop DDT
A>ED SCAN.ASM↲     Re-edit the source program, and make both changes
     : *NSUB↲
    7: *0LT↲        CTRL-Z
    7:                  SUB     C           ;LARGER VALUE IN C?
    7: *SSUB^ZCMP^Z0LT↲
    7:                  CMP     C           ;LARGER VALUE IN C?
    7: *↲
    8:                  JNC     NFOUND      ;JUMP IF LARGER VALUE NOT FOUND
    8: *SNC^ZC^Z0LT↲
    8:                  JC      NFOUND      ;JUMP IF LARGER VALUE NOT FOUND
    8: *E↲
```

```
A>ASM SCAN.AAZ↵                    Re-assemble. Selection source from disk A
                                                hex to disk A
CP/M ASSEMBLER - VER 2.0                        print to Z (selects no file)
0122
002H USE FACTOR
END OF ASSEMBLY

A>DDT SCAN.HEX↵                    Re-run debugger to check changes


DDT VERS 2.2
NEXT PC
0121 0000

-L116↵              check to ensure end is still at 116H

 0116   JMP         0000
 0119   STAX        B
 011A   NOP
 011B   INR         B

⌫  (rubout)
-G100,116↵          Go from beginning with breakpoint at end

*0116               breakpoint reached

-D121↵              look at "LARGE"
                    correct value computed
0121 06 00 22 21 00 02 7E EB 77 13 23 EB 0B 78 B1 .. '!... W .#..X.
0130 C2 27 01 C3 03 29 00 00 00 00 00 00 00 00 00 .'...)........
0140 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ...............

⌫           (rubout) abort long typeout
-G0↵        stop DDT, debug session complete
```

Notes on the Zcim Project Edition

The Zcim Project goal is to improve the accessibility of legacy Z80 CP/M era content. As part of the project, Jim Burlingame (*jb@samplx.org*) created a *Typst* version of this manual.

You can find out more about the project at its site z80cim.org. The sources of this document are available on GitHub.

The source material is from the *Tim Olmstead Memorial Digital Research CP/M Library*

The individual documents from the archive include:

- *CP/M Dynamic Debugging Tool (DDT) User's Guide* (pdf)

The contents of the manual were edited using the Typst.app site.

License